

측정은 실제로 어떻게 돌아가나

"Global 버킷 변화를 보려면 무엇을 측정하고, 버튼을 누르면 무엇이 실행돼 비교값이 나오나" — 코드와 실제 로그로.

측정의 정체는 단순하다: 리소스의 살아있는 상태(레지스트리)는 항상 메모리에 있고, 체크포인트 버튼을 누르면 그 순간의 상태를 스냅샷으로 로그에 찍는다. 두 체크포인트의 스냅샷을 **빼면** 그게 비교값이다. 이 문서는 그 상태가 어디 살고(\$1~3), 스냅샷이 어떻게 찍히며(\$4~5), 실제로 "Global/텍스처 변화"가 어떻게 측정됐는지(\$6)를 코드로 따라간다.

0 큰 그림



측정이 읽는 대상 = 가운데 두 상자(Scope의 키 목록 · Registry의 refCount). 이게 "살아있는 상태"다.

1 상태는 어디 사나 — 두 자료구조

측정할 "상태"는 딱 두 곳에 있다: Registry의 딕셔너리와 Scope의 HashSet.

Registry = 살아있는 핸들의 단일 진실. 키마다 { 핸들, refCount } 를 쥔다.

Assets/Scripts/Managers/Core/Resource/ResourceRegistry.cs

```

public sealed class Entry {
    public AsyncOperationHandle handle; // Addressables 핸들
    public int RefCount; // 이 리소스를 소유한 '서로 다른 스코프' 수
}
private readonly Dictionary<ResourceKey, Entry> _entries = new(); // ← 측정이 읽는 상태
  
```

Scope = 수명 티켓(IDisposable). 자기가 소유한 키를 HashSet 으로만 쥔다(핸들은 안 줌).

Assets/Scripts/Managers/Core/Resource/ResourceScope.cs

```

public sealed class ResourceScope : IDisposable {
    private readonly HashSet<ResourceKey> _acquired = new(); // 이 스코프가 소유한 키들
    public int Count => _acquired.Count; // ← [ResourceReport]의 global=N / scene=N 이 값
}
  
```

그래서 "GLOBAL 버킷 크기" = ?

Global 스코프의 `_acquired.Count` 그 자체다. 로그의 `global=14` 는 Global 스코프가 쥔 키가 14개라는 뜻. 버킷은 별도 컬렉션이 아니라 "어느 스코프가 그 키를 쥐고 있나"로 정해진다.

2 로드하면 refCount가 어떻게 되나

핵심: 같은 스코프가 같은 키를 또 요청해도 refCount는 안 는다 (HashSet.Add).

ResourceScope.cs - 스코프가 로드를 받으면

```
public async UniTask<T> LoadAsync<T>(string key, ...) {
    ResourceKey rk = new ResourceKey(key, typeof(T));
    bool isFirstTimeToLoad = _acquired.Add(rk); // 이 스코프에 처음이면 true, 이미 있으면 false
    return await _registry.LoadAsync<T>(rk, isFirstTimeToLoad, tok); // 그 bool을 넘긴다
}
```

ResourceRegistry.cs - 레지스트리가 실제 로드/카운트

```
public async UniTask<T> LoadAsync<T>(ResourceKey key, bool incrementRef, ...) {
    if (_entries.TryGetValue(key, out Entry entry)) { // 이미 로드돼 있음 (캐시 히트)
        if (incrementRef) entry.RefCount++; // 새 스코프가 소유하면 +1, 같은 스코프면 그대로
        return entry.handle.Result as T; // Addressables 재로드 안 함 → 핸들만 리턴
    }
    // _entries에 없음 → 여기가 '실제 Addressables 로드'의 유일한 지점
    AsyncOperationHandle handle = Addressables.LoadAssetAsync<T>(key.Key);
    entry = new Entry { handle = handle, RefCount = incrementRef ? 1 : 0 };
    _entries[key] = entry;
    ResourceMetrics.RecordLoad(key.Key, sw.ElapsedMilliseconds); // ← [LoadStats] '실제 로드'만 집계
    return handle.Result as T;
}
```

REFCOUNT = "로드 횟수"가 아니라 "소유 스코프 수"

Scene과 Dungeon 두 스코프가 같은 에셋을 쥐면 → **핸들 1개, refCount 2**. 하지만 Scene이 같은 에셋을 **3번** 요청하면 `HashSet.Add` 가 2·3번째는 false → **refCount는 1**. 그래서 `[LoadStats]` 에서 같은 키가 여러 번 찍히면 그건 "회수 후 재로드"의 직접 증거가 된다(캐시 히트는 안 세니까).

3 Dispose하면 — refCount 감소, 0이면 언로드

"누가 Release를 부르나" = 수명 경계(씬 회전·팝업 닫힘)가 스코프를 Dispose한다. 수동 호출 없음.

ResourceScope.cs - 스코프가 죽으면 쥔 키를 전부 반납

```
public void Dispose() {
    foreach (ResourceKey key in _acquired) _registry.Release(key); // 쥔 키를 하나씩 반납
    _acquired.Clear();
}
```

ResourceRegistry.cs - refCount가 0이 되면 그때 진짜 언로드

```
public void Release(ResourceKey key) {
    if (!_entries.TryGetValue(key, out Entry entry)) return;
    if (--entry.RefCount ≤ 0) { // 마지막 소유자가 손 뗐
        if (entry.handle.IsValid()) Addressables.Release(entry.handle); // ← 여기서 메모리 회수
        _entries.Remove(key);
        OnReleased?.Invoke(key); // 파생물(아틀라스 캐시)도 같이 버리라 통지
    }
}
```

이것이 §6 회수의 실체

던전으로 씬이 바뀌면 GameScene의 Scene 스코프가 `Dispose()` → 팝업 키들이 `Release` → refCount 0 → `Addressables.Release` → **텍스처 168.8 → 142.1MB로 실제 반납**. 개발자가 Release를 부른 곳은 없다.

4 "측정" = 이 상태를 스냅샷으로 찍는 것

두 종류를 찍는다: 버킷 핸들(무엇이 살아있나) + 메모리 총량(얼마 쓰나).

(A) 버킷 스냅샷 — 레지스트리를 훑어 스코프별 카운트 + 키 목록을 로그로.

ResourceManager.cs - LogAliveReport (→ [ResourceReport])

```
public void LogAliveReport(string tag) {
    _registry.GetSnapshot(_regSnapshot); // 살아있는 핸들 전부
    sb.Append($"[ResourceReport] ({tag}) handles={_registry.Count}");
    foreach (var kv in _scopes) // 스코프별 Count = global=14 scene=17 ...
        sb.Append($" {kv.Value.Name}={kv.Value.Count}");
    foreach (var e in _regSnapshot) // 각 키를 어느 버킷이 쥐었나 + refCount + 출처
        sb.AppendLine($" [{ResolveBucket(e.Key)}] {e.Key.Key} (ref {e.RefCount}) ← {source}");
    GameLog.Log(sb.ToString()); // → 태그 필터 → 파일
}
```

(B) 메모리 스냅샷 — 빌드에선 런타임 안전 총량을 찍는다(에디터 ×2 사본 없음).

ResourceMetrics.cs - LogRuntimeMemory (→ [ResourceMemory])

```
public static void LogRuntimeMemory(string label) {
    GameLog.Log($"[ResourceMemory] ({label}) " +
        $"총 할당 {ToMB(Profiler.GetTotalAllocatedMemoryLong())} | " +
        $"그래픽스 {ToMB(Profiler.GetAllocatedMemoryForGraphicsDriver())} | " +
        $"텍스처 {ToMB((long)Texture.currentTextureMemory)}"); // ← 회수 판정의 주 신호
}
```

5 버튼(체크포인트)을 누르면 실행되는 것

F5~F9 한 번 = 구분선 + 버킷 스냅샷 + 메모리 스냅샷을 한 순간에 기록.

MetricsHotkeys.cs - 빌드에서 F5~F9

```
private static void Mark(string label) {
    ResourceMetrics.Checkpoint(label); // [Checkpoint] — label — (구분선)
    Managers.Resource.LogAliveReport(label); // [ResourceReport] 버킷 핸들 수 + 목록
    ResourceMetrics.LogRuntimeMemory(label); // [ResourceMemory] 총 할당/텍스처
}
// Update()에서: f5 → Mark("1. 진입 직후"), f6 → Mark("2. 팝업..."), ... f9 → LogLoadStats()+Mark("5. 종료")
```

핵심

버튼은 새 값을 만들지 않는다. 이미 살아있는 상태(\$1~3)를 그 순간 스냅샷으로 찍을 뿐. 그래서 "① 누른 스냅샷"과 "② 누른 스냅샷"을 빼면 그 사이 조작(팝업 열기)의 효과가 그대로 비교값이 된다.

6 워크스루 — "Global/텍스처 변화"를 이렇게 측정했다

실제 세션 로그(metrics_20260811_004330.log)로 단계별로. ← 는 그 줄이 무슨 뜻인지.

STEP 1 · F5 → 기준선 스냅샷 (①)

```
[Checkpoint] — 1. GameScene 진입 직후 —
[ResourceReport] (1. 진입) handles=32 global=14 scene=17 party=6 atlasSprites=36
[Global] SceneTable / Fire / Droid_Helmet_AR (ref 3) / ... / UI_LootNotification ← Global 14개의 정체
[ResourceMemory] (1. 진입) 총 할당 332.9MB | 그래픽스 471.2MB | 텍스처 140.7MB
```

기록된 기준값: Global 핸들 14개, 텍스처 140.7MB. — 이게 "before"다.

STEP 2 · 팝업 5종 열기 (플레이어 조작 — 코드가 자동으로 로드)

팝업을 열면 내부적으로 `UIManager.ShowPopupUIAsync` → `Scope.LoadAsync` → `Registry` 가 돌아 §2대로 **Scene** 스코프에 팝업 키들이 쌓인다. (개선 전이라면 `isGlobal:true` 라 Global에 쌓였을 것 — 그 한 줄 차이가 리팩터의 전부.)

STEP 3 · F6 → 팝업 세션 스냅샷 (②)

```
[ResourceReport] (2. 팝업 첫 오픈) handles=46 global=15 scene=31 party=6 atlasSprites=255
[Scene] UI_EquipmentUpgradePanel ← UIManager.ShowPopupUIAsync ← 팝업이 Scene에 쌓임(회수 가능)
[Global] EquipmentIconAtlas ← UI_LootNotification.CreateToast... ← Global 14→15 (loot 토스트, 의도된 것)
[ResourceMemory] (2. 팝업) 총 할당 337.1MB | 텍스처 168.8MB
```

비교값 = ② - ①: 텍스처 **168.8 - 140.7 = +28.1MB** (팝업 세션 비용). Scene 17→31(팝업들), Global은 +1(loot 아틀라스)뿐.

STEP 4 · 던전 왕복 (씬 회전 = Scene 스코프 Dispose → §3 Release)

GameScene → BossDungeon으로 씬이 바뀌면 GameScene의 Scene 스코프가 `Dispose()` → 팝업 키들 `Release` → `refCount 0` → `Addressables.Release`. 수동 호출 0줄, 씬 경계가 자동으로.

STEP 5 · F8 → 왕복 후 스냅샷 (④)

```
[ResourceReport] (4. 던전 왕복) handles=33 global=15 scene=16 party=6
[ResourceMemory] (4. 왕복) 총 할당 343.9MB | 텍스처 142.1MB
```

비교값 = ④ - ②: 텍스처 **168.8 → 142.1 = -26.7MB 회수** (진입 140.7 수준 복귀). Scene 버킷도 31→16으로 팝업 반납. = 누수 없음이 텍스처 회수로 실증.

정리 — "비교값으로 측정했다"의 실체

① 스냅샷과 ②·④ 스냅샷은 같은 상태(레지스트리·텍스처 메모리)를 다른 시점에 찍은 것일 뿐이다. 그 차이가 곧 "팝업 세션 +28.1MB", "왕복 회수 -26.7MB"라는 비교값. 버튼은 값을 만들지 않고 시점을 고정할 뿐이며, 값을 움직인 건 그 사이의 조작(팝업 열기·씬 회전)과 그걸 처리하는 스코프/refCount 구조다.

지표	어디서 나오나	비교 방법
Global 버킷 크기	[ResourceReport] 의 <code>global=N</code> (= Global 스코프 <code>_acquired.Count</code>)	체크포인트 간 N 비교 (14 → 15)
Global이 무엇으로 찾나	그 아래 [Global] key ← source 목록	목록 diff (누가 새로 상주?)
메모리 회수 여부	[ResourceMemory] 의 텍스처	①↔②↔④ 차이 (+28 → -26.7)
회수 후 재로드 비용	[LoadStats] 키별 로드 횟수	2회 이상 = 재로드(대가)